

Unlocking Forest Secrets: K-Nearest Neighbors (KNN) for Forest Cov- ertype Classification (July 2024)

Aayush Man Shrestha¹ and Pradeepti Dongol²

Department of Electronics and Computer Engineering, IOE, Thapathali Campus, Kathmandu 44600, Nepal

Corresponding author: Aayush Man Shrestha (aayush.m.shrestha@gmail.com)

ABSTRACT Classifying Forest covertypes is essential for effective environmental management and conservation. This work explores the application of K-Nearest Neighbors (KNN), an instance-based learning algorithm, for this task. We delve into the core principles of KNN, including the selection of distance metrics and the impact of the "K" value on classification accuracy. This report utilizes the Forest Covertypes dataset available at the UCI Machine Learning repository to train and evaluate KNN models for predicting forest cover types. We investigate the performance of KNN with different values of K, various distance metrics, and feature selection techniques to identify the most effective configuration for this dataset. The results demonstrate the adaptability and robustness of KNN in ecological data analysis.

INDEX TERMS Classification, Distance metrics, Forest covertypes, Instance-based, KNN

I. INTRODUCTION

K-Nearest Neighbors (KNN) is a simple yet powerful algorithm widely used for various classification and regression tasks, including ecological data analysis. Its popularity stems from its intuitive approach, ease of implementation, and robust performance across diverse datasets. KNN has been effectively applied in numerous fields such as medical diagnosis, image recognition, and ecological classification, making it a versatile tool in the data scientist's toolkit.

Instance-Based Learning: KNN is an instance-based learning algorithm, meaning it makes predictions based on the similarity between new data points and previously observed instances. It operates on the principle that similar observations are likely to belong to the same class. This is particularly relevant in ecological data analysis, where environmental variables often exhibit distinct patterns that can be leveraged

for classification. In the context of forest covertypes classification, KNN identifies the most similar forest types based on the provided features, such as soil type, elevation, and climate conditions. This approach allows KNN to effectively capture the complex relationships between different environmental factors and forest types.

Distance Metrics: A key component of KNN is the distance metric used to measure similarity between instances. Commonly used metrics include Euclidean distance, Manhattan distance, and Minkowski distance. The choice of distance metric can significantly impact the algorithm's performance. For example, Euclidean distance is suitable for continuous features, while Manhattan distance is more appropriate for categorical features. Additionally, other distance metrics like Hamming distance can be employed for binary or categorical data. The flexibility in choosing a distance metric

allows KNN to be tailored to the specific characteristics of the dataset, enhancing its predictive accuracy.

K Value Selection: The "K" in KNN represents the number of nearest neighbors considered when making a prediction. Choosing the right value for K is crucial for the algorithm's accuracy. A small K value might lead to overfitting, where the model is too sensitive to noise in the data. Conversely, a large K value might result in underfitting, where the model oversimplifies the data patterns. Cross-validation techniques are often used to determine the optimal K value for a given dataset. This involves partitioning the data into training and validation sets multiple times to evaluate the performance of different K values, ensuring that the chosen K value generalizes well to unseen data.

KNN's ability to adapt to different types of data and its straightforward implementation make it a popular choice for various applications. By understanding and leveraging its key components, such as distance metrics and K value selection, practitioners can harness the full potential of KNN to solve complex classification tasks, including the classification of forest cover types.

The advantages of KNN are:

- **Simplicity and Ease of Implementation:** KNN is straightforward to implement and understand, making it an excellent choice for beginners and quick prototyping.
- **No Training Phase:** Unlike many other algorithms, KNN does not require a training phase. It stores the entire training dataset and makes predictions based on real-time comparisons.
- **Flexibility:** KNN can handle both classification and regression tasks, making it a versatile tool for various applications.
- **Adaptability to New Data:** KNN easily incorporates new data points without retraining the model. This is particularly useful in dynamic environments where data evolves over time.

The limitations of KNN are:

- **Computational Complexity:** KNN can be computationally intensive, especially with large datasets. The need to calculate distances for every data point can slow down predictions.
- **Storage Requirements:** Since KNN stores all training data, it can require significant storage space for large datasets.
- **Sensitivity to Irrelevant Features:** The presence of irrelevant or redundant features

can negatively impact KNN's performance. Feature selection and normalization are essential preprocessing steps.

- **Class Imbalance:** KNN can struggle with imbalanced datasets where some classes are underrepresented.

In this lab report, we explore the application of KNN for classifying forest cover types. By leveraging the inherent strengths of KNN and addressing its limitations, we aim to achieve accurate and insightful classifications that contribute to ecological understanding and conservation efforts.

II. RELATED WORK

The application of K-nearest neighbors (KNN) for classification tasks has been extensively studied across various domains. This section reviews key studies and methodologies that have utilized KNN, particularly in the context of ecological and forest covertype classification.

KNN in Ecological Data Analysis: KNN has been widely adopted in ecological data analysis due to its simplicity and effectiveness. One prominent study by Peterson et al. (2011) demonstrated the use of KNN for species distribution modeling. The researchers highlighted KNN's capability to handle high-dimensional ecological data and its robustness in predicting species' habitats based on environmental variables [1]. Similarly, Guisan et al. (2007) employed KNN to model plant species richness, showcasing its adaptability to various types of ecological datasets and its capacity to provide accurate predictions [2].

Forest Covertype Classification: Forest covertype classification is a crucial task in environmental monitoring and management. Blackard and Dean (1999) conducted one of the foundational studies in this area, utilizing KNN to classify forest cover types in the Roosevelt National Forest. Their study underscored KNN's effectiveness in distinguishing between different forest types based on features such as elevation, soil type, and canopy cover [3]. More recent research by Kavzoglu and Mather (2002) compared the performance of KNN with other classification algorithms, including decision trees and support vector machines. Their findings indicated that KNN, when combined with appropriate distance metrics and feature selection techniques, could achieve competitive accuracy in forest covertype classification [4].

Advancements in Distance Metrics: The choice of distance metric significantly impacts KNN's performance. Studies have explored various distance metrics to optimize KNN for specific applications. For instance, Liu et al. (2016) introduced a hybrid distance metric combining Euclidean and Hamming distances for mixed-type data, which improved classification accuracy in heterogeneous datasets [5]. Another study by Zhang et al. (2018) proposed an adaptive distance metric that adjusts weights based on feature relevance, enhancing KNN's predictive power for complex ecological data [6].

Feature Selection and Dimensionality Reduction: Effective feature selection is critical for improving KNN's performance, particularly in high-dimensional datasets. Guyon and Elisseeff (2003) reviewed various feature selection techniques, emphasizing their importance in reducing computational complexity and enhancing classification accuracy [7]. Their work provides a comprehensive overview of methods such as recursive feature elimination and mutual information-based selection, which are applicable to KNN. In the context of forest covertype classification, Zhu et al. (2017) demonstrated the use of principal component analysis (PCA) to reduce dimensionality and improve KNN's efficiency and accuracy [8].

Handling Imbalanced Datasets: Class imbalance is a common challenge in ecological and forest covertype datasets, where certain classes are underrepresented. Japkowicz and Stephen (2002) provided a detailed analysis of techniques to address class imbalance, including oversampling, undersampling, and synthetic data generation methods such as SMOTE (Synthetic Minority Over-sampling Technique) [9]. These techniques have been successfully applied to KNN, as demonstrated by Chawla et al. (2002), who showed that incorporating SMOTE with KNN significantly improved classification performance in imbalanced datasets [10].

The studies indicate that KNN is a versatile and effective algorithm for ecological and forest covertype classification. Its performance can be further enhanced through the careful selection of distance metrics, feature selection techniques, and methods to address class imbalance metrics.

III. METHODOLOGY

A. DATASET DESCRIPTION

K-Nearest Neighbors (KNN) is a machine learning algorithm that classifies data points based on their similarity to neighboring points in a feature space. By calculating distances to its nearest neighbors in the feature space, KNN determines the most likely class label for each instance, aligning its predictions with the dataset's objective.

The Covertype database is used to predict forest cover type based on cartographic variables. The dataset is a set of raw unscaled data collected from four wilderness study areas with minimal human-caused disturbances in the Roosevelt National Forest of northern Colorado: Rawah, Neota, Comanche Peak, and Cache la Poudre.

Neota has the highest mean elevation value and contains spruce/fir tree species while Cache la Poudre has the lowest mean elevation value and has Ponderosa pine, Douglas-fir, and cottonwood/willow as the major tree species. Rawah and Cache la Poudre, that have intermediate values of mean elevation and mostly contain lodgepole pine, spruce/fir, and aspen tree species. The dataset mostly contains data of Rawah and Comanche Peak due to tree species assortment and range of predictive variable values while the most unique class is Cache la Poudre due to its relatively low elevation range and species composition.

The objective of the dataset is to classify the data into one of seven possible predominant types of vegetation or forest cover type.

The dataset consists of 12 cartographic measures derived from data originally obtained from the US Geological Survey (USGS) and USFS data while the target variable or actual forest cover type was determined from US Forest Service (USFS) Region 2 Resource Information System (RIS) data. 10 out of 12 measures are quantitative variables while the remaining two attributes: wilderness area and soil type variables are converted into 4 and 40 binary data respectively using one-hot encoding method resulting in a total of 54 features. The target consists of seven major forest cover types spruce/fir, lodgepole Pine, Ponderosa Pine, Cottonwood/Willow, Aspen, Douglas-fir and Krummholz represented by integers 1 to 7.

There are 581012 instances in the dataset, where each data point represents the cover type for 30x30 meter cells.

The dataset is stored in covetype.csv and contains the following attributes

1. Elevation

It is a quantitative data type that represents elevation of the 30x30 meter cell in meters

2. Aspect

It is a quantitative data type that measures aspect in azimuth

3. Slope

It is a quantitative data type that measures slope of 30x30 meter cell in degrees.

4. Horizontal_Distance_To_Hydrology

It is a quantitative data type that measures the horizontal distance to nearest surface water features in meters.

5. Vertical_Distance_To_Hydrology

It is a qualitative data type that measures the vertical distance to nearest surface water features in meters.

6. Horizontal_Distance_To_Roadways

It is a qualitative data type that measures the horizontal distance to nearest roadway in meters

7. Hillshade_9am

It is a qualitative data type that measures the hillshade index at 9am, summer solstice. It is represented by an index ranging from 0 to 255.

8. Hillshade_Noon

It is a qualitative data type that measures the hillshade index at noon, summer solstice. It is represented by an index ranging from 0 to 255.

9. Hillshade_3pm

It is a qualitative data type that measures the hillshade index at 3pm, summer solstice. It is represented by an index ranging from 0 to 255.

10. Horizontal_Distance_To_Fire_Points

It is a qualitative data type that measures the horizontal distance to nearest wildfire ignition points in meters.

11. Wilderness_Area (4 binary columns)

It is categorical data containing 4 classes one-hot encoded to 4 binary columns where 0 indicates absence and 1 indicates presence. This feature represents the wilderness area designation.

12. Soil_Type (40 binary columns)

It is categorical data type with 40 classes one-hot encoded to 40 binary columns where absence and presence is indicated by 0 and 1 respectively. This attribute represents the soil type designation.

13. Cover_Type (7 types)

It is a categorical data type with 7 classes representing 7 forest cover type designation indicated by integers 1 to 7.

The target variable Cover_Type consists of 7 classes denoted by integers 1 to 7. The class and its respective integer value is:

1. Spruce/Fir

2. Lodgepole Pine

3. Ponderosa Pine

4. Cottonwood/Willow

5. Aspen

6. Douglas-fir

7. Krummholz

B. PROPOSED METHODOLOGY

The Covertype dataset used in this report contains integer attributes along with encoded categorical attributes making it ideal for computation of distance.

The computation of K-nearest neighbor involves several steps.

The preprocessing begins with checking whether the dataset contains any missing values in order to either discard the attribute or instance or compute the appropriate value. Since the dataset doesn't contain missing values, missing value handling need not be performed.

The Covertype dataset contains 10 integer features and 2 categorical features that are already one hot encoded. Hence, data type conversion is not necessary.

So, the dataset is normalized to standardize numerical features. After that, the feature importance analysis and selection is performed using One-Way ANOVA Test, Chi-Square Test, Mutual Information, Correlation Test and PCA which results in dropping of features having low correlation with the target.

After relevant features are selection, the distribution of target variable is checked to ensure uniform distribution.

Once data preprocessing is completed, the data is split into train, validation and test set in 70:15:15 ratio.

The KNN model is built on the dataset using different distance metrics like Manhattan distance, Euclidean distance and Chebyshev distance. The appropriate value of K is selected for each distance metric producing a complete KNN model.

Once trained, the model is evaluated using accuracy, precision, recall and F1-score. Confusion matrix and is used to visualize the model's performance.

Similar process was applied after dimensionality reduction using PCA where the model was evaluated for 5 and 10 principal components using Euclidean distance.

Additionally, the KNN model was trained for weighted distance using Manhattan and Euclidean distance.

Algorithm: K-Nearest Neighbor

1. Import necessary libraries
 2. Load data from CSV file
 3. Perform feature importance analysis using One-Way ANOVA Test, Chi-Square Test, Mutual Information, Correlation Test and Principal Component Analysis
 4. Drop irrelevant features
 5. Check for imbalanced data
 6. Split the data into training and test sets
 7. Determine the most appropriate value of K
 8. Train KNN model using Manhattan distance, Euclidean distance and Chebyshev distance for default and weighted distance metrics
 9. Predict target variables for the test set
 10. Evaluate classifier performance
-

C. MATHEMATICAL FORMULAE

The K-Nearest Neighbors was trained used the following formulas:

One-Way ANOVA Test Formula:

For source of variation = “Within”, the sum of squares (SSW), degrees of freedom (df_w) and Means Squares (MSW) are as follows:

$$SSW = \sum_{j=1}^k \sum_{i=1}^l (X - \bar{X}_j)^2 \quad (1)$$

$$df_w = k - 1 \quad (2)$$

$$MSW = \frac{SSW}{df_w} \quad (3)$$

For source of variation = “Between”, the sum of squares (SSB), degrees of freedom (df_b) and Means Squares (MSB) are as follows:

$$SSB = \sum_{j=1}^k (\bar{X}_j - \bar{X})^2 \quad (4)$$

$$df_b = n - k \quad (5)$$

$$MSW = \frac{SSB}{df_b} \quad (6)$$

where, X represents observations in the dataset, k is the number of groups of the independent variables and n is the total number of observations across all groups.

A. Shrestha and P. Dongol: KNN Classifier

The F-score is given by:

$$F = \frac{MSB}{MSW} \quad (7)$$

Chi-Square Test:

$$\chi_c^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (8)$$

where, c, O and E represents degrees of freedom, observed values and expected values respectively.

Mutual Information:

$$MI = \sum_{a,b} P(a_i, b_j) \log\left(\frac{P(a_i, b_j)}{P(a_i)P(b_j)}\right) \quad (9)$$

where, a_i and b_j represent random variables, for $i, j = 1$ to n , $P(a_i, b_j)$ is the joint probability distribution of a_i and b_j , and $P(a_i)$ and $P(b_j)$ are the marginal probability distributions of a_i and b_j respectively.

Correlation Test:

$$r_{xy} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \quad (10)$$

where, r_{xy} represents the correlation coefficient between x and y , x_i and y_i represents the values of X and Y within a sample, and $\bar{x}$ and $\bar{y}$ represents the average values of X and Y within a sample.

PCA:

Let's assume we have a matrix or dataset with dimensional $m \times n$.

The formula for the mean (μ) is:

$$\mu = \frac{\sum x_{ij}}{m \times n} \quad (11)$$

where, x_{ij} represents the element at the i th row and j th column of the matrix.

The formula for the covariance matrix Σ is:

$$\Sigma = \frac{1}{m - 1} (X^T \cdot X) \quad (12)$$

For a square matrix A, the eigenvalues can be obtained by solving the characteristic equation:

$$|A - \lambda I| = 0 \quad (13)$$

where, λ is the eigenvalue and I is the identity matrix of the same size as A.

Given its eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ the eigenvectors can be found by solving the equation:

$$(A - \lambda_i I)v_i = 0 \quad (14)$$

The proportion of variance (PoV) for each eigenvalue λ_i is given by

$$PoV_i = \frac{\lambda_i}{\sum_{j=1}^n \lambda_j} \quad (15)$$

where, $\sum_{j=1}^n \lambda_j$ represents the sum of all eigenvalues.

Distance Metrics:

The Minkowski distance is given by:

$$d = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \quad (16)$$

where, i represents the number of dimensions, p represents the parameter that determines the type of distance metric, and x and y are the two points between which the distance is being calculated.

The Manhattan distance ($p = 1$) is given by:

$$d = |x_1 - x_2| + |y_1 - y_2| \quad (17)$$

The Euclidean distance ($p = 2$) is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (18)$$

The Chebyshev distance ($p \rightarrow \infty$) is given by:

$$d = \max(|x_A - x_B|, |y_A - y_B|) \quad (19)$$

The formulae used for performance evaluation are:

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (20)$$

where, TP (True Positives) represents correctly predicted positive instances, TN (True Negatives) represents correctly predicted negative instances, FP (False Positives) represents incorrectly predicted positive instances and FN (False Negatives) represent incorrectly predicted negative instances.

Precision:

$$Precision = \frac{TP}{TP + FP} \quad (21)$$

Recall:

$$Recall = \frac{TP}{TP + FN} \quad (22)$$

F1-score:

$$F1 \text{ Score} = 2 \times \frac{Precision * Recall}{Precision + Recall} \quad (23)$$

where, Precision is the ratio of correctly predicted positive instances to the total predicted positive instances and Recall ratio represents the correctly predicted positive instances to the total actual positive instances.

D. INSTRUMENTATION DETAILS

The analysis and visualization in this study were conducted using Jupyter Notebook, a versatile and interactive web-based coding environment that allows for the integration of code execution, rich text, mathematics, plots, and rich media in a single document. This platform was chosen for its ease of use and its extensive support for data science and machine learning workflows.

The core libraries used in this study were NumPy, Pandas, Scikit-learn, Matplotlib, and Seaborn. NumPy is a fundamental library for scientific computing in Python, providing support for large multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays. In this study, NumPy was used for various numeric operations, such as array manipulations and mathematical computations that are integral to the preprocessing and analysis of data.

Pandas is essential for data manipulation and analysis, offering data structures and functions needed to manipulate structured data seamlessly. Pandas was utilized for importing the dataset, handling missing values, and performing Exploratory Data Analysis (EDA). The library's DataFrame structure was particularly useful for organizing the data and preparing it for analysis. Scikit-learn, a robust machine learning library in Python, provides simple and efficient tools for data mining and data analysis. It was used extensively in this study. The `train_test_split` function was employed to split the dataset into training and testing sets, essential for evaluating the model's performance. Preprocessing tools such as `StandardScaler`, `OneHotEncoder`, and `LabelEncoder` were used to normalize and encode the data.

The `f_oneway` function from `scipy.stats` was used for conducting one-way ANOVA tests to determine significant differences between the means of multiple groups. The `chi2_contingency` function from `scipy.stats` was employed for conducting chi-squared tests to assess the association between categorical variables. From `statsmodels.stats.outliers_influence`, the `variance_inflation_factor` function was used to assess multicollinearity among variables, providing insights into potential issues that could affect model performance and interpretation. This function from `sklearn.feature_selection` was used to compute the mutual information between features and the

target variable, aiding in feature selection based on their relevance to the prediction task.

Evaluation metrics including accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_curve, and auc were used to calculate and assess the performance of the models.

For data visualization, Matplotlib and Seaborn were employed. Matplotlib provides a comprehensive plotting framework, while Seaborn builds on Matplotlib to offer a high-level interface for drawing attractive and informative statistical graphics. These libraries were used to create visual representations of the confusion matrix, aiding in the interpretation of the classifier's performance, and to generate histogram plots to visualize the distribution of data and results.

IV. EXPERIMENTAL RESULTS

Feature Selection: We explored different techniques, such as One-Way ANOVA test, Chi-Square test of association, Mutual information, Correlation test, and Principal Component Analysis (PCA) for feature selection and dimensionality reduction.

1. One-Way ANOVA Test

We performed One-way ANOVA test between numerical features and our target categorical feature "Cover_Type", which has 7 categories. The results, presented in Table 1, indicate that all p-values are zero.

2. Chi-Square Test

We performed a Chi-Square Test of Association to examine the relationship between categorical features and the target categorical feature "Cover_Type," which has 7 categories. Due to the large number of categorical variables, presenting the results in a table was not feasible. The summary of the results is as follows: the degrees of freedom (dof) was 6, and all p-values were less than 0.5.

3. Mutual Information

We selected ten features with strong dependencies on the target class using the 'mutual_info_classif' function from the sklearn package. The ten features and their respective scores are presented in Table 2. Subsequently, we calculated the Variance Inflation Factor (VIF) values for the selected features, as shown in Table 3. All VIF values were low, ranging between 1 and 2, indicating minimal multicollinearity.

4. Correlation Test

First, the correlation of the features with the target variable was calculated and plotted as shown in Figure 1. The features with the highest correlation with the target feature were Wilderness_Area4, Elevation, Soil_Type10, and Wilderness_Area1. These features had correlation values (absolute values) greater than 0.20. Wilderness_Area4 had the highest positive correlation and Elevation had the highest negative correlation. In contrast, the features with the lowest correlation with the target feature were the features describing soil types 16, 18, 25, 15, 9, 34, 8, 28, 7, and 26. These features had correlation values less than 0.01.

Next, the correlation among the features themselves was calculated. All the features exhibited correlation values below 0.80 with each other. The pair with the highest correlation was Wilderness_Area1 and Wilderness_Area4, showing a negative correlation of 0.79.

5. PCA

PCA was performed on the dataset, yielding the explained variance ratios for the top ten principal components as follows: 0.2254, 0.1957, 0.1521, 0.1008, 0.0688, 0.0508, 0.0406, 0.0337, 0.0285, and 0.0255. With ten principal components, approximately 90% of the variance in the dataset was preserved. In contrast, using only five principal components preserved about 75% of the variance.

Check if the data is unbalanced: The dataset was found to be unbalanced, as illustrated in Figure 2. There was a significant class imbalance, where Cover types 1 and 2 constituted the majority, while Cover types 4, 5, and 6 were represented by a much smaller proportion. Covertype 1 (36.46%) and Covertype 2 (48.76%) collectively accounted for almost 85% of the total instances in the dataset. Covertype 4 (0.47%), Covertype 5 (1.63%) and Covertype 6 (2.99%) collectively accounted for only 5% of the total instances in the dataset. Likewise, Covertype 3 (6.15%) and Covertype 7 (3.53%) collectively accounted for almost 10% of the total instances in the dataset. So, to mitigate this class imbalance, stratification was performed while splitting the data into training and testing sets. This helps to ensure that the class proportions are maintained in both the training and testing sets and reduces the risk of the model becoming biased toward the majority class.

K value selection: From [Figure 3](#), we observe that the values of K ranging from 3 to 7 yield good performance, with balanced training and testing accuracy. To select an optimal K that is neither too small nor too large, we choose $K = 5$. This choice also aligns with the default setting of the KNN classifier in the scikit-learn package.

Effect of Distance metrics: The model was evaluated using three different distance metrics: Manhattan distance, Euclidean distance and Chebyshev distance. The KNN classifier was initialized and trained for each of the three distance metrics and the model was evaluated using confusion matrix ([Figure 4](#), [Figure 5](#) and [Figure 6](#)) and classification report ([Table 4](#), [Table 5](#) and [Table 6](#)).

First, the evaluation was performed for KNN using the **Manhattan distance**. The diagonal elements of the confusion matrix, that indicate the number of correct predictions for each class, yielded higher values than the off-diagonal elements, that indicate the number of incorrect predictions for each class. Covertype2 and Covertype1 have relatively high correct predictions, with 81063 and 59271 instances respectively, but still show some misclassifications. The highest number of misclassifications occurs when cover type 1 is predicted as cover type 2 and when cover type 2 is predicted as cover type 1, with a total of 3778 and 2175 instances respectively. From the classification report for Manhattan distance, the classification report indicates a strong overall performance, with an accuracy of 94%. The model performs particularly well on the largest classes, cover types 1, 2, and 7, achieving precision, recall, and F1-scores around 0.93 to 0.95. Cover types 3 and 6 also show high performance with F1-scores of 0.92 and 0.83, respectively. However, the model struggles more with the less represented cover types 4 and 5, reflected in lower recall and F1-scores of around 0.79 to 0.82.

Then the model was evaluated for KNN using **Euclidean distance**. The resulting confusion matrix indicated strong performance for most classes. Cover type 1 and Cover type 2 were predicted with high accuracy, of 58,710 and 80480 correct predictions respectively. However, the misclassification for Cover type 1 as Cover type 2 was 4,292 while that of cover type 2 as cover type 1 was 3,661. Cover type 3 and Cover type 6 showed good accuracy with most

predictions correct, though Class 3 had 559 instances misclassified into Class 6. Cover type 4 and 5 had fewer instances but demonstrated more misclassifications. Class 7 had 5,733 correct predictions with minimal confusion. The classification report indicated a total accuracy of 93%. The precision, recall, and F1-scores for Cover type 1, 2 and 7 were observed to be particularly high, consistently around 0.93 to 0.95, while the model faces challenges with less prevalent classes, such as Cover type 4 and 5, where precision and recall drop to around 0.76 to 0.85, indicating some misclassifications. Cover type 3 and 6 indicated intermediate results with the precision, recall and f1-score ranging from 0.90 to 0.91 and 0.81 to 0.83 respectively. The macro average F1-score of 0.88 and the weighted average F1-score of 0.93 reflected the dominance of more frequent classes.

Similarly, for the KNN model built using **Chebyshev distance**, the confusion matrix indicates high performance for most classes with the highest correct predictions of 79589 and 57830 for cover type 2 and 1 respectively. However, these two classes also contain the highest misclassification of 5102, where cover type 1 was classified as cover type 2, and 4330, where cover type 2 was misclassified as cover type 1. Overall, the Chebyshev distance metric yields result comparable to Manhattan and Euclidean metrics, with high accuracy for dominant classes. Similar to previous distance metrics, the highest results are produced by cover type 1, 2 and 7 with the precision, recall and f1-score ranging from 0.91 to 0.94. Cover type 3 indicated intermediate results with precision 0.88, recall and f1-score of 0.89. However, the score dropped to the range of 0.72 to 0.76 recall, 0.79 to 0.83 precision for cover type 3, 4, and 5. The total accuracy of the model was found to be 91% with macro average and weighted average F1-score of 0.85 and 0.91 respectively.

Effect of Dimensionality Reduction: First, using the dataset after dimensionality reduction using PCA with the **number of components = 5**, a KNN classifier was initialized and trained. The performance of the model was evaluated using a confusion matrix ([Figure 7](#)) and classification report ([Table 7](#)). The classification metrics for each covertype reveal a comprehensive evaluation of the model's performance, culminating in an overall accuracy of 81%.

Covertime 1 demonstrated a precision of 0.81, recall of 0.82, and an F1-score of 0.81, showcasing robust performance. Covertime 2, the largest category, exhibited a precision of 0.83, a recall of 0.85, and an F1-score of 0.84, indicating strong classification capabilities. Covertime 3 achieved a precision of 0.78, recall of 0.77, and an F1-score of 0.77, showing reliable performance in its classification. Cotypes 4 and 5 demonstrated lower recall values (0.61 and 0.44) but maintained reasonable precision and F1 scores of approximately 0.67 and 0.54, respectively. Covertime 6 exhibited a precision of 0.67, a recall of 0.56, and an F1-score of 0.61, indicating satisfactory performance. Covertime 7 showed strong precision (0.85), recall (0.73), and an F1-score (0.78), highlighting effective classification. Overall, the model achieved a balanced accuracy of 81%.

Then, PCA was performed with the **number of components = 10**. Using this dataset, another KNN classifier was initialized and trained. The performance of the model was evaluated using a confusion matrix (Figure 8) and classification report (Table 8). The classification performance of the model, as detailed by the precision, recall, and F1-score for each covertime, indicates a strong overall accuracy of 91%. Covertime 1 achieved a precision of 0.92, recall of 0.91, and an F1-score of 0.92, demonstrating excellent performance. Covertime 2, the largest category, also performed well with a precision of 0.92, a recall of 0.94, and an F1-score of 0.93. Covertime 3 showed good results with a precision of 0.87, recall of 0.88, and an F1-score of 0.88. Cotypes 4 and 5, despite having fewer instances, had lower recall values (0.72 and 0.71) but still maintained reasonable precision and F1-scores around 0.76. Covertime 6 had a precision of 0.79, a recall of 0.74, and an F1-score of 0.77, indicating a slightly lower but acceptable performance. Finally, Covertime 7 achieved high precision (0.94), recall (0.91), and an F1-score (0.93). Overall, the model demonstrates consistent performance across most cotypes, particularly excelling in those with larger support, while maintaining a balanced accuracy of 91%.

Effects of Weighted Distance Metrics: Similar to the default distance metrics, the model was trained for using weighted distance metrics for Manhattan and Euclidean distance and the evaluation was done using

confusion matrix (Figure 9 and Figure 10) and classification report (Table 9 and Table 10).

The confusion matrix for weighted **Manhattan distance** shows that Cover Type 1 and 2 have highest correct predictions (59455 and 81255 respectively) with relatively few misclassifications whereas Cover type 4 and 5 have the lowest correct prediction of 672 and 2325 instances respectively. From the classification report, it was observed that the algorithm achieved an overall accuracy of 94%. It demonstrates strong performance across all seven forest cover types, with precision, recall, and F1-scores consistently high for Cover type 1, 2 and 7 ranging from 0.94 to 0.96. Cover type 3 and 6 also show good performance with F1-scores around 0.92 to 0.86. However, cover type 4 and 5 exhibit slightly lower F1-scores of 0.84 and 0.83 respectively, indicating some difficulty in correctly predicting these less frequent cover types. The macro average F1-score was 0.90 while the weighted average F1-score was 0.94.

Similarly, for the KNN model trained using weighted **Euclidean distance** metric, the confusion matrix indicated strong accuracy for Cover type 1 and 2 with highest correct predictions of 58990 and 80731 respectively. However, the misclassification for these classes is also high with 4024 cover type 1 predicted as cover type 2 and 3386 cover type 2 predicted as cover type 1. Cover type 4 had the least misclassification with 2, 119 and 53 instances misclassified as cover type 2, 3 and 6 respectively. However, cover type 4 and 5 were often misclassified into other classes that are more pronounced. The classification report for this model depicted a total accuracy of 93% with macro average and weighted average f1-score of 0.89 and 0.93 respectively. Cover type 1, 2, 3 and 7 performed well with precision, recall and f1-score ranging from 0.91 to 0.95 while that of cover type 4, 5 and 6 ranged from 0.79 to 0.85.

V. DISCUSSION AND ANALYSIS

Feature Selection: Different feature selection and dimensionality reduction techniques were explored to identify the most important features and eliminate the irrelevant ones.

For One-Way ANOVA and Chi-square tests, the hypotheses are as follows:

Null Hypothesis (H₀): The means of the qualitative feature are the same across all 7

different class subgroups, indicating that this feature cannot differentiate between the subgroups and cannot predict class types.

Alternative Hypothesis (H1): The means of the qualitative feature differ across at least one of the 7 class subgroups, indicating that this feature can differentiate between the subgroups and can predict class types.

In the One-Way ANOVA test, all the p-values of the numerical features were less than 0.5, which indicates that there is statistically significant evidence to reject the null hypothesis. This suggests that at least one of the numerical features varies significantly across the different class subgroups, making them potentially useful for predicting or distinguishing between these subgroups.

Similarly, in the Chi-square test, all the p-values of the categorical features were less than 0.5, which indicates significant evidence against the null hypothesis. This suggests that there are associations or dependencies between the categorical features and the target variable, highlighting their potential importance in predicting or classifying different outcomes.

Next, using Mutual Information, the dependency between the variables was measured. The top 10 features with the highest mutual information with the target class were identified using the `mutual_info_classif` function from the sklearn package. Subsequently, all these features were found to have low VIF values, indicating minimal multicollinearity among them. In the correlation test, the features having a very low correlation value (less than a threshold of 0.01) with the target class were dropped from the dataset. The dropped features include Soil_Type16, Soil_Type18, Soil_Type25, Soil_Type15, Soil_Type9, Soil_Type34, Soil_Type8, Soil_Type28, Soil_Type7, and Soil_Type26. Likewise, features having a very high correlation with each other (greater than a threshold of 0.80) were dropped from the dataset. However, no such features were identified, eliminating the need for further feature dropping.

Using PCA, dimensionality reduction was employed to extract the most relevant features while preserving variance in the dataset. This reduction in dimensions not only enhances computational efficiency but also aids in mitigating the curse of dimensionality, potentially improving the performance of subsequent machine learning models by focusing on the most informative features.

Additionally, the choice between five and ten principal components reflects a trade-off between model complexity and retained variance. Using five principal components retains approximately 75% of the variance in the dataset, indicating a substantial amount of information retained while effectively reducing the dimensionality. Using ten principal components allows for the retention of an additional 15% variance of the dataset, but this comes with an increase in computational complexity.

K value selection: To determine the optimal K, we consider both performance metrics and potential classification issues such as overfitting and underfitting. Choosing a low value of K can lead to overfitting because the model may be too sensitive to noise in the training data, capturing details that do not generalize well to new data. Conversely, choosing a high value of K can lead to underfitting because the model becomes too generalized, smoothing over important patterns in the data. After evaluating various K values, we determined that K=5 strikes a balance between overfitting and underfitting. This value of K is optimal as it minimizes classification errors by adequately capturing the underlying patterns in the data without being overly sensitive to noise. Additionally, K=5 aligns with the default setting in the scikit-learn KNN classifier, which is a widely accepted and tested configuration in the machine learning community.

Effect of Distance Metrics: The K-Nearest Neighbors (KNN) model was tested using three different distance metrics: Manhattan, Euclidean, and Chebyshev. Each metric gave insights into how well the model classified forest cover types. Manhattan distance performed best with 94% accuracy, showing high precision and recall for main classes like Cover types 1, 2, and 7 with F1-scores around 0.93 to 0.95. However, it struggled more with less common classes (Cover types 4 and 5), which had lower recall and F1-scores around 0.79 to 0.82. Euclidean distance achieved 93% accuracy, performing similarly well for major classes but finding it harder to distinguish between less represented cover types. Chebyshev distance also scored 91% accuracy, performing well for main classes but had more misclassifications between Cover types 1 and 2. These results highlight how different distance metrics affect the KNN model's ability to classify forest cover types.

effectively. On comparing other critical performance metrics, Manhattan distance consistently achieved the highest precision, recall, and F1-scores for major forest cover types (1, 2, and 7), reflecting its robust performance in accurately classifying these classes. Euclidean distance showed comparable precision and recall but struggled slightly more with less represented classes (4 and 5), while Chebyshev distance, though effective overall, exhibited notable misclassifications between Cover types 1 and 2, impacting its precision and recall slightly more in those cases.

Effect of Dimensionality Reduction: Using Principal Component Analysis (PCA) to reduce the dimensionality of the dataset yielded insightful results. When PCA was applied with 5 principal components, it managed to retain 75% of the variance present in the original dataset. This dimensionality reduction led to a good classification performance, achieving an accuracy of 81%. To explore the impact of retaining more variance, PCA was then applied with 10 principal components, capturing approximately 92% of the dataset's variance. This increase in retained variance had a significant positive effect on the model's performance, resulting in an impressive accuracy of 91%. The 10% improvement in accuracy demonstrates the importance of retaining a higher amount of variance for better classification performance. Moreover, this enhancement was not limited to accuracy alone. Other critical performance metrics such as precision, recall, and F1 score also saw substantial improvements of approximately 10% across all classes. This indicates a more robust and reliable classification model, with better capability to distinguish between different classes in the dataset.

Effects of Weighted Distance Metrics: The KNN models using weighted distance metrics were trained using Manhattan and Euclidean distance. Both models exhibited strong performance with overall accuracies of 94% and 93%, respectively. Weighted Manhattan distance showed robust precision, recall, and F1-scores across all major classes (Cover types 1, 2, and 7), with values ranging from 0.94 to 0.96. It struggled slightly more with less represented classes (Cover types 4 and 5), achieving lower F1-scores around 0.84 and 0.83. Similarly, weighted Euclidean distance performed well for dominant classes but faced

challenges in accurately predicting Cover types 4 and 5, resulting in lower precision and recall of approximately 0.79 to 0.85. Overall, both metrics demonstrated effective classification capabilities for the forest cover types, emphasizing the impact of weighted distances on model performance across varying class distributions.

The weighted Manhattan and weighted Euclidean showed improved precision, recall, and F1-scores for major classes (Cover types 1, 2, and 7), with values consistently around 0.91 to 0.96, similar to their respective non-weighted counterparts. However, the weighted metrics exhibited slightly enhanced performance in distinguishing between less prevalent classes (Cover types 4 and 5), achieving F1-scores of approximately 0.83 to 0.84, compared to 0.79 to 0.82 for the default metrics. independence.

VI. CONCLUSION

In conclusion, we thoroughly explored the K-Nearest Neighbors (KNN) algorithm for classifying forest cover types, focusing on feature selection, distance metrics, and dimensionality reduction techniques. Our analysis underscored the significance of appropriate feature selection methods such as One-Way ANOVA, Chi-square tests, Mutual Information, and VIF analysis in identifying the most informative features while eliminating irrelevant ones. The One-Way ANOVA and Chi-square tests revealed statistically significant differences in feature means across class subgroups, highlighting the importance of both numerical and categorical features in predicting class types.

We tested various K values to balance overfitting and underfitting, ultimately determining that K=5 provided optimal performance, minimizing classification errors and adequately capturing underlying data patterns. The evaluation of different distance metrics—Manhattan, Euclidean, and Chebyshev—revealed that the Manhattan distance metric achieved the highest accuracy at 94%, particularly excelling in precision and recall for major cover types such as Cover types 1, 2, and 7. However, it struggled with less common classes, similar to the Euclidean and Chebyshev metrics, which achieved slightly lower accuracies and faced challenges in distinguishing less represented cover types. The impact of dimensionality reduction using Principal Component Analysis (PCA) was also

significant. Applying PCA with five principal components retained 75% of the dataset's variance, resulting in an accuracy of 81%. Increasing the number of principal components to ten captured 92% of the variance, substantially improving the model's performance to 91% accuracy. This enhancement was accompanied by notable improvements in precision, recall, and F1 scores across all classes, demonstrating the critical role of retaining sufficient variance for robust classification. Moreover, the use of weighted distance metrics further improved the model's performance. The weighted metrics also exhibited better performance in distinguishing less prevalent classes compared to their non-weighted counterparts.

Overall, our investigation highlighted the importance of careful feature selection, appropriate K value determination, and the choice of distance metrics in optimizing KNN classifier performance. Future research could explore advanced techniques for further enhancing classification accuracy and addressing the challenges of class imbalance and feature correlation, thereby improving predictive capabilities in diverse applications.

APPENDIX

A. Feature Analysis

Table 1 Results on One-Way ANOVA Test

Features	F statistic	p value	Significant
Elevation	155306.87	0.0	True
Horizontal Distance To Roadways	10275.78	0.0	True
Slope	7762.48	0.0	True
Horizontal Distance To Fire Points	7262.06	0.0	True
Hillshade 9am	3231.99	0.0	True
Hillshade Noon	2920.39	0.0	True
Horizontal Distance To Hydrology	2364.95	0.0	True
Vertical Distance To Hydrology	1205.33	0.0	True
Hillshade 3pm	1042.19	0.0	True
Aspect	446.72	0.0	True

Table 2 Showing Mutual Information Score

Columns	Score
Elevation	0.457276
Wilderness Area4	0.146321
Horizontal Distance To Roadways	0.089416
Wilderness Area1	0.076449
Horizontal Distance To Fire Points	0.070076
Soil Type10	0.062986
Slope	0.036356
Soil Type29	0.033301
Hillshade 9am	0.032389
Soil Type38	0.030759

Table 3 Showing VIF Values

Features	vif value
const	130.221636
Elevation	1.966047
Wilderness Area4	1.896948
Wilderness Area1	1.869536
Horizontal Distance To Roadways	1.516337
Soil Type29	1.467779
Soil Type10	1.414355
Horizontal Distance To Fire Points	1.248293
Slope	1.238711
Hillshade 9am	1.189322
Soil Type38	1.080894

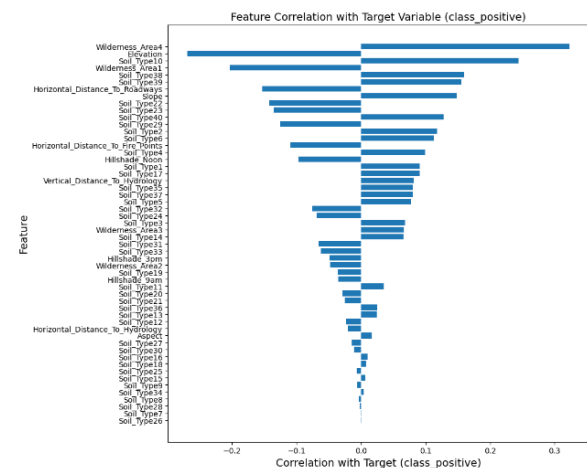


Figure 1 Feature Correlation with Target Variable

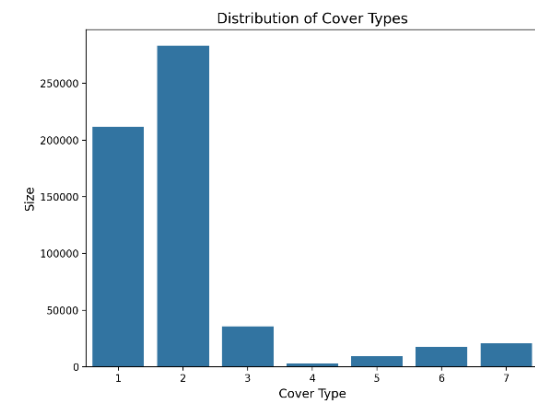


Figure 2 Class Distribution

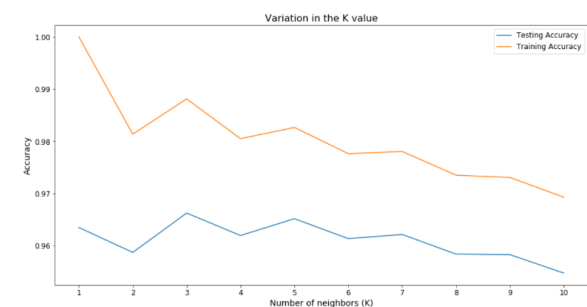


Figure 3 Variation of Accuracy due to Variation of K value

B. KNN using Default Distance Metrics

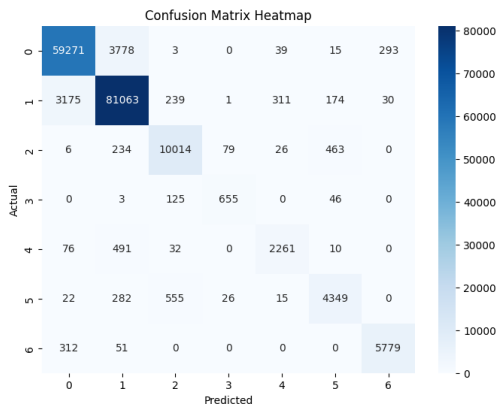


Figure 4 Confusion matrix for KNN using Manhattan distance

Table 4 Classification Report for KNN using Manhattan distance

Coverttype	Precision	Recall	F1-score	Support
1	0.94	0.93	0.94	63399
2	0.94	0.95	0.95	84993
3	0.91	0.93	0.92	10822
4	0.86	0.79	0.82	829
5	0.85	0.79	0.82	2870
6	0.86	0.83	0.84	5249
7	0.95	0.94	0.94	6142
Accuracy			0.94	174304
Macro avg	0.90	0.88	0.89	174304
Weighted avg	0.94	0.94	0.94	174304

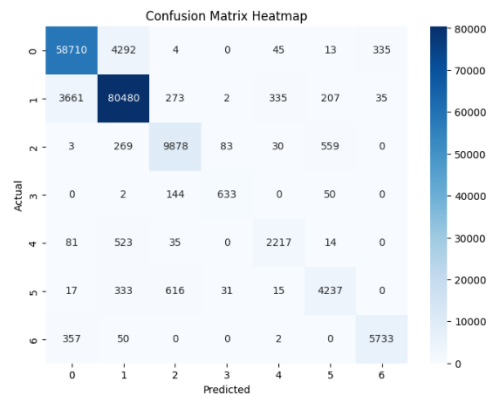


Figure 6 Confusion Matrix for KNN using Euclidean distance

Table 5 Classification Report for KNN using Euclidean distance

Coverttype	Precision	Recall	F1-score	Support
1	0.93	0.93	0.93	63399
2	0.94	0.95	0.94	84993
3	0.90	0.91	0.91	10822
4	0.85	0.76	0.80	829
5	0.84	0.77	0.80	2870
6	0.83	0.81	0.82	5249
7	0.94	0.93	0.94	6142
Accuracy			0.93	174304
Macro avg	0.89	0.87	0.88	174304
Weighted avg	0.93	0.93	0.93	174304

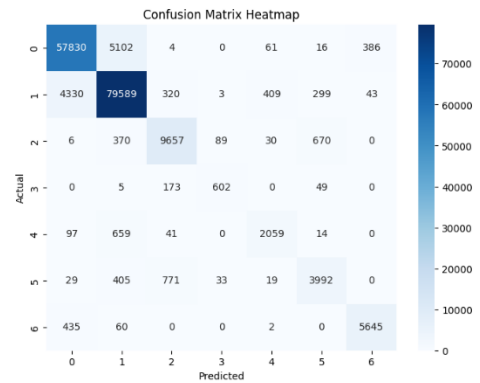


Figure 7 Confusion Matrix for KNN using Chebyshev distance

Table 6 Classification Report for KNN using Chebyshev distance

Coverttype	Precision	Recall	F1-score	Support
1	0.92	0.91	0.92	63399
2	0.92	0.94	0.93	84993
3	0.88	0.89	0.89	10822
4	0.83	0.73	0.77	829
5	0.80	0.72	0.76	2870
6	0.79	0.76	0.78	5249
7	0.93	0.92	0.92	6142
Accuracy			0.91	174304
Macro avg	0.87	0.84	0.85	174304
Weighted avg	0.91	0.91	0.91	174304

C. Effect of Dimensionality Reduction

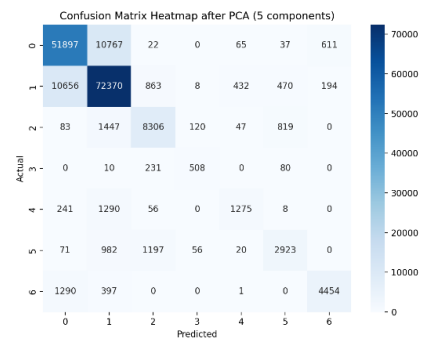


Table 7 Classification Report for KNN using 5 Principal Components

Figure 5 Confusion Matrix for KNN using 5 Principal Components

Coverttype	Precision	Recall	F1-score	Support
1	0.81	0.82	0.81	63399
2	0.83	0.85	0.84	84993
3	0.78	0.77	0.77	10822
4	0.73	0.61	0.67	829
5	0.69	0.44	0.54	2870
6	0.67	0.56	0.61	5249
7	0.85	0.73	0.78	6142
Accuracy			0.81	174304
Macro avg	0.77	0.68	0.72	174304
Weighted avg	0.81	0.81	0.81	174304

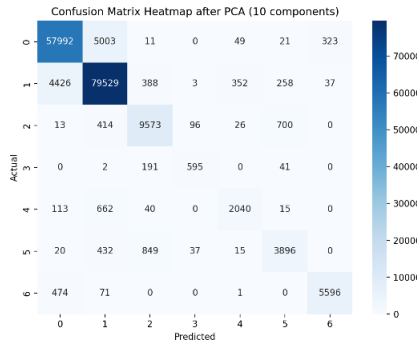


Figure 8 Confusion Matrix for KNN using 10 Principal Components

Table 8 Classification Report for KNN using 10 Principal Components

Coverttype	Precision	Recall	F1-score	Support
1	0.92	0.91	0.92	63399
2	0.92	0.94	0.93	84993
3	0.87	0.88	0.88	10822
4	0.81	0.72	0.76	829
5	0.82	0.71	0.76	2870
6	0.79	0.74	0.77	5249
7	0.94	0.91	0.93	6142
Accuracy			0.91	174304
Macro avg	0.87	0.83	0.85	174304
Weighted avg	0.91	0.91	0.91	174304

D. KNN using Weighted Distance Metrics

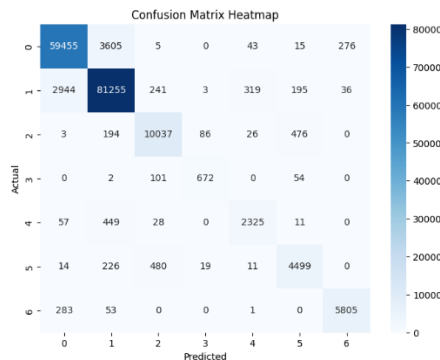


Figure 9 Confusion Matrix for KNN using weighted Manhattan distance

Table 9 Classification Report for KNN using weighted Manhattan distance

Coverttype	Precision	Recall	F1-score	Support
1	0.95	0.94	0.94	63399
2	0.95	0.96	0.95	84993
3	0.92	0.93	0.92	10822
4	0.86	0.81	0.84	829
5	0.85	0.81	0.83	2870
6	0.86	0.86	0.86	5249
7	0.95	0.95	0.95	6142
Accuracy			0.94	174304
Macro avg	0.91	0.89	0.90	174304
Weighted avg	0.94	0.94	0.94	174304

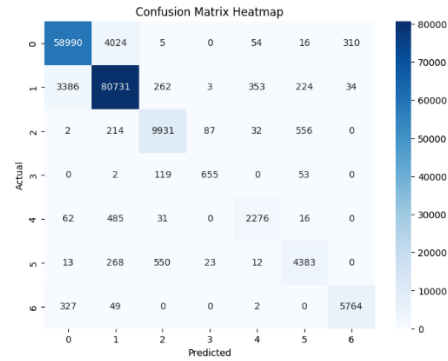


Figure 10 Confusion Matrix for KNN using weighted Euclidean distance

Table 10 Classification Report for KNN using weighted Euclidean distance

Coverttype	Precision	Recall	F1-score	Support
1	0.94	0.93	0.94	63399
2	0.94	0.95	0.95	84993
3	0.91	0.92	0.91	10822
4	0.85	0.79	0.82	829
5	0.83	0.79	0.81	2870
6	0.84	0.84	0.84	5249
7	0.94	0.94	0.94	6142
Accuracy			0.93	174304
Macro avg	0.89	0.88	0.89	174304
Weighted avg	0.93	0.93	0.93	174304

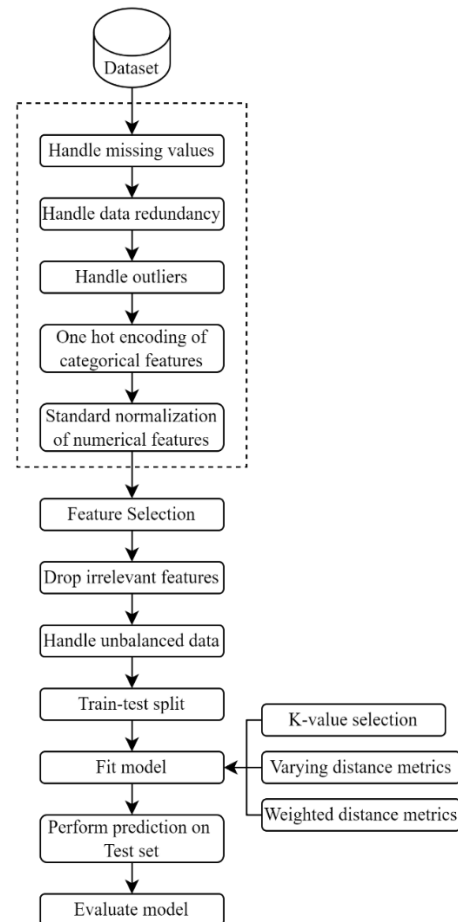


Figure 11 System Block Diagram

ACKNOWLEDGEMENT

We extend our heartfelt appreciation to the Department of Electronics and Computer Engineering at IOE, Thapathali Campus for providing us with the invaluable opportunity to engage in learning, research, and the realization of our insights. We would like to express our sincere gratitude to our esteemed lecturer Er. Dinesh Baniya Kshatri for providing unwavering support, invaluable guidance, and insightful suggestions throughout our academic career.

REFERENCES

- [1] A. T. Peterson, M. Papes and M. Eaton, "Transferability and model evaluation in ecological niche modeling: a comparison of GARP and Maxent," *Ecography*, vol. 30, no. 4, pp. 550-560, 2011.
- [2] A. Guisan, T. C. Edwards Jr and T. Hastie, "Generalized linear and generalized additive models in studies of species distributions: setting the scene," *Ecological Modelling*, vol. 157, no. 2-3, pp. 89-100, 2002.
- [3] J. A. Blackard and D. J. Dean, "Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables," *Computers and Electronics in Agriculture*, vol. 24, no. 3, pp. 131-151, 1999.
- [4] T. Kavzoglu and P. M. Mather, "The role of feature selection in artificial neural network applications," *International Journal of Remote Sensing*, vol. 23, no. 15, pp. 2919-2937, 2002.
- [5] Y. Liu, T. Xiao and W. Zhou, "A hybrid distance metric for heterogeneous data," *Knowledge-Based Systems*, vol. 93, pp. 66-74, 2016.
- [6] H. Zhang, A. C. Berg, M. Maire and J. Malik, "SVM-KNN: Discriminative nearest neighbor classification for visual category recognition," *In 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, vol. 2, pp. 2126-2136, 2006.
- [7] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, no. Mar, pp. 1157-1182, 2003.
- [8] X. Zhu and A. B. Goldberg, "Introduction to semi-supervised learning," *Synthesis Lectures on Artificial Intelligence and Machine Learning*, vol. 3, no. 1, pp. 1-130, 2009.
- [9] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," *Intelligent Data Analysis*, vol. 6, no. 5, pp. 429-449, 2002.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321-357, 2002.

BIOGRAPHY



Aayush Man Shrestha

is currently pursuing his undergraduate degree in Computer Engineering at the Institute of Engineering (IOE), Thapathali Campus. His research interests span across diverse areas of ML and AI, including natural language processing, computer vision, robotics, and automation.



Pradeepti Dongol

is currently pursuing her undergraduate degree in Computer Engineering at the Institute of Engineering (IOE), Thapathali Campus. Her research interests span across diverse areas, including natural language processing, generative AI, arts and design and Large Language Models (LLMs).